

Dynamic Hand Gesture Recognition Using LSTM and MediaPipe

Technical Report

Name: Kirubel Sentayehu, GSR/7879/17

Course: Computer Vision

Date: 9/9/2025

Executive Summary

This project implements a real-time dynamic hand gesture recognition system using Long Short-Term Memory (LSTM) networks and Google's MediaPipe framework. The system successfully recognizes 6 different hand gestures with 83% accuracy by processing temporal sequences of hand landmarks extracted from video frames.

Key Achievements:

- Developed an end-to-end gesture recognition pipeline
- Achieved 83% validation accuracy on 6 gesture classes
- Implemented real-time inference with Streamlit web application
- Optimized processing with GPU acceleration using CuPy

1. Introduction

Problem Definition

Dynamic hand gesture recognition involves identifying human hand movements that change over time by analyzing sequences of video frames. Unlike static gesture recognition, this task requires understanding temporal patterns and motion dynamics.

Examples of dynamic gestures:

- 🙌 Waving
- ➡️ Swiping left/right
- 🛑 Stop sign with motion

Challenge

Dynamic gestures are more challenging than static ones because they require:

- Temporal modeling of sequential data
- Understanding motion patterns across multiple frames
- Real-time processing capabilities

2. Dataset

20BN-Jester Dataset

- **Total Videos:** 64,448 gesture videos
- **Original Classes:** 27 unique gesture types
- **Resolution:** 100×176 pixels, ~~37 frames per video~~ (3 seconds)

Selected Subset (Resource Constraints)

Due to computational limitations, we selected 6 most common gesture classes:

Class ID	Gesture Name	Training Videos	Validation Videos
0	Doing Other Things	~2,240	~327
10	Sliding Two Fingers Down	~2,240	~327
11	Sliding Two Fingers Left	~2,240	~327
12	Sliding Two Fingers Right	~2,240	~327
13	Sliding Two Fingers Up	~2,240	~327
14	Stop Sign	~2,240	~327

Total: 13,402 training videos, 1,953 validation videos

3. Methodology

3.1 System Architecture

The system follows a three-stage pipeline:

```
Video Frames → MediaPipe Hand Detection → LSTM Classification → Gesture Prediction
```

3.2 MediaPipe Hand Landmark Extraction

Why Use Hand Landmarks?

- **Low-dimensional:** 21 landmarks × 3 coordinates = 63 features (vs 52,800 raw pixels)
- **Invariant:** Not sensitive to lighting, background, scale, or rotation
- **Robust & Fast:** Reliable detection with minimal computational cost

Hand Landmark Model: MediaPipe detects 21 key points on the hand:

- Wrist (1 point)
- Thumb (4 points)
- Index finger (4 points)
- Middle finger (4 points)
- Ring finger (4 points)
- Pinky finger (4 points)

Each landmark provides (x, y, z) coordinates, creating a 63-dimensional feature vector.

3.3 Data Preprocessing Pipeline

Step 1: Frame-to-Landmark Conversion

```
def frame_to_landmark_vec_gpu(bgr_image, landmark_detector):
    # Convert BGR to RGB
    rgb_image = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)

    # Detect hand landmarks
    detection_result = landmark_detector.detect(mp_image)
```

```

if detection_result.hand_landmarks:
    landmarks = detection_result.hand_landmarks[0]
    return landmarks_to_vector(landmarks) # Shape: (63,)
else:
    return np.zeros(63, dtype=np.float64) # No hand detected

```

Step 2: Temporal Sequence Normalization

- All video sequences normalized to exactly 30 frames
- Longer sequences: Uniform sampling
- Shorter sequences: Zero padding
- Final shape: (30, 63) per video

Step 3: GPU Acceleration

- Used CuPy for GPU-accelerated array operations
- Parallel processing with ThreadPoolExecutor
- Batch processing for efficiency

3.4 LSTM Model Architecture

```

model = Sequential([
    Input(shape=(30, 63)),           # 30 frames, 63 features each
    Masking(mask_value=0),           # Ignore padded frames
    LSTM(128, return_sequences=True), # First LSTM layer
    Dropout(0.2),                    # Regularization
    LSTM(64),                        # Second LSTM layer
    Dropout(0.2),                    # More regularization
    Dense(32, activation='relu'),     # Feature compression
    Dense(6, activation='softmax')    # 6-class classification
])

```

Architecture Rationale:

- **Two LSTM layers:** Capture multi-level temporal patterns
- **Masking layer:** Handles variable-length sequences elegantly
- **Dropout (0.2):** Prevents overfitting
- **Progressive compression:** 128 → 64 → 32 → 6 features

3.5 Training Configuration

- **Optimizer:** Adam with default learning rate
- **Loss Function:** Sparse categorical crossentropy
- **Batch Size:** 32
- **Early Stopping:** Patience of 3 epochs
- **Maximum Epochs:** 20

4. Implementation Details

4.1 Technical Stack

- **Deep Learning:** TensorFlow/Keras
- **Computer Vision:** MediaPipe, OpenCV

- **GPU Acceleration:** CuPy
- **Web Interface:** Streamlit
- **Data Processing:** NumPy, Pandas

4.2 Real-time Inference Pipeline

Sliding Window Approach:

```
frame_buffer = deque(maxlen=30) # Circular buffer

def predict_gesture(new_frame):
    landmark_vec = extract_landmarks(new_frame)
    frame_buffer.append(landmark_vec)

    if len(frame_buffer) == 30:
        sequence = np.array(frame_buffer)[None, :, :]
        prediction = model.predict(sequence)
        return get_gesture_name(prediction)
```

4.3 Web Application

- **Framework:** Streamlit with streamlit-webrtc
- **Features:**
 - Real-time camera feed
 - Live gesture prediction
 - Confidence scoring
 - Performance metrics display

5. Results

5.1 Training Performance

Training Progress:

- **Initial Accuracy:** ~52% (Epoch 1)
- **Final Training Accuracy:** ~83% (Epoch 11)
- **Final Validation Accuracy:** ~81% (Epoch 11)
- **Early Stopping:** Triggered at Epoch 11 due to no improvement

Loss Curves:

- Training loss decreased from 1.34 to 0.49
- Validation loss decreased from 0.71 to 0.55
- No significant overfitting observed

5.2 Classification Report

Gesture	Precision	Recall	F1-Score	Support
Doing Other Things	0.78	0.93	0.85	713
Sliding Two Fingers Down	0.81	0.84	0.82	249
Sliding Two Fingers Left	0.84	0.71	0.77	246

Sliding Two Fingers Right	0.86	0.69	0.77	242
Sliding Two Fingers Up	0.93	0.68	0.79	244
Stop Sign	0.91	0.93	0.92	259

Overall Performance:

- **Accuracy:** 83%
- **Macro Average F1-Score:** 0.82
- **Weighted Average F1-Score:** 0.83

5.3 Performance Analysis

Best Performing Gestures:

1. **Stop Sign:** 92% F1-score (most distinctive)
2. **Doing Other Things:** 85% F1-score (largest dataset)

Challenging Gestures:

- Sliding gestures (Left/Right/Up) show lower recall
- Similar motion patterns cause confusion between directional swipes

5.4 Real-time Performance

- **Target FPS:** 30 fps
- **Actual Performance:** ~20-25 fps
- **Processing Time:** ~40-50ms per frame
- **Bottleneck:** MediaPipe hand detection (~30ms)

6. System Deployment

6.1 Web Application Features

- **Real-time Camera Feed:** Live gesture recognition
- **Confidence Threshold:** Adjustable sensitivity
- **Frame Buffer Visualization:** Shows processing status
- **Screenshot Capability:** Save predictions for analysis

6.2 User Interface

- Clean, intuitive design
- Real-time feedback
- Performance metrics display
- Configuration controls in sidebar

7. Discussion

7.1 Strengths

- ✓ **Robust Feature Extraction:** Hand landmarks are invariant to lighting and background
- ✓ **Temporal Modeling:** LSTM effectively captures gesture dynamics
- ✓ **Real-time Performance:** Suitable for interactive applications
- ✓ **GPU Optimization:** Efficient processing with CuPy acceleration
- ✓ **Scalable Architecture:** Can extend to more gesture classes

7.2 Limitations

- ⚠️ **Hand Detection Dependency:** Fails when MediaPipe can't detect hands
- ⚠️ **Single Hand Only:** Limited to one hand gestures
- ⚠️ **Class Imbalance:** Some gestures under-represented in dataset
- ⚠️ **Directional Confusion:** Similar sliding motions are hard to distinguish
- ⚠️ **Computational Requirements:** Needs GPU for optimal performance

7.3 Comparison with Alternatives

vs. CNN-based approaches:

- ✅ More efficient (63 features vs thousands of pixels)
- ✅ Better temporal modeling with LSTM
- ❌ Dependent on hand detection quality

vs. Traditional computer vision:

- ✅ More robust to variations
- ✅ End-to-end learning
- ❌ Requires large dataset

8. Conclusion

This project successfully demonstrates dynamic hand gesture recognition using modern deep learning techniques. The combination of MediaPipe's robust hand detection and LSTM's temporal modeling capabilities resulted in an effective system achieving 83% accuracy on 6 gesture classes.

Key Contributions

1. **Efficient Pipeline:** MediaPipe landmarks reduce dimensionality by 99.88%
2. **Temporal Modeling:** LSTM architecture captures gesture dynamics
3. **Real-time System:** Deployable web application with live inference
4. **GPU Optimization:** CuPy acceleration for improved performance

Future Work

1. **Expand Dataset:** Include all 27 gesture classes
2. **Two-Hand Support:** Extend to multi-hand gestures
3. **Data Augmentation:** Improve robustness with synthetic data
4. **Model Optimization:** Explore lightweight architectures for mobile deployment
5. **Advanced Architectures:** Investigate Transformer models for sequence processing

Project Repository: <https://github.com/kira-1011/DynamicHandRecognition>

Demo Application: <https://dynamichandrecognition.streamlit.app/>